

SUPPLY MANAGEMENT SYSTEM

Task Addendum

Multi-Tenancy Architecture

Maps to FSD Addendum 25 | ASP.NET Core 8 | EF Core 8 | SQL Server 2022

Version 1.0 | 2026

FSD source	FSD Addendum 25 - Multi-Tenancy Architecture
Total tasks	8
Total estimated dev days	24 days (~5 working weeks)
Critical priority tasks	6
Total unit test cases	49
Epics	4 epics: tenant foundation, isolation, UI/workflow, migration
Key architecture	Shared DB, org_id discriminator, EF Core HasQueryFilter, zero-change regression for e

1. FSD Traceability Summary

Every task maps to a specific section of FSD Addendum 25.

Task ID	Title	Days	FSD Ref	Priority
MT-001	tenant_schema, Organizations, FeatureDefinitions, and Organi...	3d	Sec 2, 3.1, 3.2	Critical
MT-002	Organization CRUD API and Feature Configuration screen	3d	Sec 2.2, 3.4	Critical
MT-003	Add organization_id to all tenant-scoped tables and apply EF...	5d	Sec 5.1, 4.2	Critical
MT-004	TenantMiddleware, FeatureAuthorizationFilter, and JWT organi...	3d	Sec 4.1, 4.2	Critical
MT-005	React sidebar rendering driven by organization features and ...	3d	Sec 4.1, 4.3	High
MT-006	Organization-scoped workflow definitions and default seeding	2d	Sec 6	High
MT-007	Designate Super Admin, migrate existing data, and create the...	2d	Sec 7	Critical
MT-008	Full regression test suite and multi-tenant integration test...	3d	Sec 7 (Step 8)	Critical
TOTAL	8 tasks	24d		

2. Detailed Task Register

Epic 1 - Tenant Foundation: Schema, Organizations, Features

tenant_schema, Organizations CRUD, FeatureDefinitions catalog, OrganizationFeatures toggle mapping, and the Super Admin management screens.

MT-001

Critical

3d

Sec 2, 3.1, 3.2

tenant_schema, Organizations, FeatureDefinitions, and OrganizationFeatures

Create the tenant_schema and all four foundation entities. Seed the feature catalog with every module/screen/capability in the system. Seed the 'Supply Chain Demo' organization with all features enabled.

Development tasks

- Create tenant_schema with TenantDbContext
- Add Organizations entity (14 fields per FSD Section 2.1)
- Add FeatureDefinitions entity (7 fields per FSD Section 3.1). Seed all features: MODULE_PROCUREMENT, MODULE_INVENTORY, MODULE_WAREHOUSE, MODULE_FINANCE, MODULE_MIR, SCREEN_SUPPLIER_SCORECARD, FEATURE_WHATSAPP, FEATURE_MASTER_LEDGERS, etc. Mark User Management and Role Management as is_core = true.
- Add OrganizationFeatures mapping entity (6 fields per FSD Section 3.2)
- Seed 'Supply Chain Demo' organization: org_code='SCM-DEMO', plan=ENTERPRISE, all OrganizationFeatures enabled
- Seed default feature templates for BASIC/STANDARD/ENTERPRISE plans per FSD Section 3.3 matrix
- EF Core migration: dotnet ef migrations add InitialTenantSchema

Unit test cases

- After migration, SCM-DEMO organization exists with is_active = true
- All FeatureDefinitions rows are seeded with correct feature_codes
- OrganizationFeatures for SCM-DEMO has is_enabled = true for every feature
- User Management and Role Management are marked is_core = true and cannot be disabled
- Feature dependency: enabling MODULE_MIR auto-enables MODULE_INVENTORY if not already enabled
- BASIC plan template does not include MODULE_FINANCE; ENTERPRISE includes everything

Acceptance criteria

- All tenant foundation tables created
- SCM-DEMO org seeded with full feature access
- Core features correctly protected from disabling

MT-002

Critical

3d

Sec 2.2, 3.4

Organization CRUD API and Feature Configuration screen

Build the Super Admin organization management API and the feature toggle configuration screen with dependency auto-enable.

Development tasks

Unit test cases

• POST /api/system/organizations - creates org + initial Admin user in one transaction. Admin receives email invitation to set password. • GET /api/system/organizations - paginated list (Super Admin only) • PUT /api/system/organizations/{id} - update settings • PATCH /api/system/organizations/{id}/deactivate - sets is_active=false, invalidates all active sessions via bulk UserSessions delete • GET /api/system/organizations/{id}/features - list features with enabled/disabled status • PUT /api/system/organizations/{id}/features - accepts feature toggle updates; validates is_core features cannot be disabled; auto-enables dependencies; rejects if disabling a feature that another enabled feature depends on • React: System Administration > Organizations screen with create/edit/deactivate. Feature Configuration sub-screen with grouped toggles. • GET /api/system/features - full FeatureDefinitions catalog for the config UI	• POST organization creates both the org and the Admin user atomically • Deactivating an org invalidates all its users' active sessions immediately • PUT features rejecting an attempt to disable User Management (is_core = true) • PUT features enabling MODULE_MIR auto-enables MODULE_INVENTORY • PUT features disabling MODULE_INVENTORY when MODULE_MIR depends on it is rejected with a dependency error • Non-Super-Admin calling any /api/system/* endpoint returns 403
Acceptance criteria	• Org creation with initial Admin is atomic • Feature dependencies enforced on enable and disable • Deactivation immediately blocks all logins for that org

Epic 2 - Tenant Isolation: organization_id, Query Filters, Middleware	Add organization_id to all tenant-scoped tables, implement EF Core global query filters, TenantMiddleware, FeatureAuthorizationFilter, and JWT claim update.
--	--

MT-003	Critical	5d	Sec 5.1, 4.2	Add organization_id to all tenant-scoped tables and apply EF Core global query filters
---------------	-----------------	-----------	---------------------	---

The largest single migration in this project. Add organization_id (UNIQUEIDENTIFIER NOT NULL, DEFAULT = SCM-DEMO UUID) to every business data table across all DbContexts. Apply HasQueryFilter on every tenant-scoped entity.

Development tasks	Unit test cases
• Add organization_id column to all tables listed in FSD Section 5.1: auth_schema (Users, Roles, RolePermissions), suppliers_schema, demand_schema, procurement_schema, inventory_schema, warehouse_schema, material_schema, finance_schema, workflow_schema • Each migration uses DEFAULT = SCM-DEMO's organization_id so every existing row automatically belongs to Supply Chain Demo • Add non-clustered index on organization_id in each table • Create ITenantContext interface: OrganizationId (Guid), IsSuperAdmin (bool) • Create ITenantScopedEntity interface: OrganizationId property, implemented by every tenant-scoped entity • In each DbContext.OnModelCreating: modelBuilder.Entity().HasQueryFilter(e => e.OrganizationId == _tenantContext.OrganizationId) for every ITenantScopedEntity • Override SaveChangesAsync in each DbContext: auto-stamp OrganizationId on new entities from ITenantContext • Verify lookups_schema: add is_global flag to LookupValues; global lookups (currencies, UOMs) exempt from tenant filter; custom lookups scoped	• After migration, every existing row in Users has organization_id = SCM-DEMO UUID • Query for Products without explicit org filter returns only SCM-DEMO products (query filter active) • Creating a new Product auto-stamps the current tenant's organization_id without explicit code • Super Admin context (IsSuperAdmin = true) bypasses the query filter and sees all orgs' data • Two organizations with the same product_code: each sees only their own, no collision • Global lookup values (currencies) visible to all orgs; custom lookups scoped correctly
Acceptance criteria	• All business tables have organization_id with no nulls • EF Core query filters active on every tenant entity • Auto-stamping on create works without business code changes • Super Admin bypass works for cross-tenant queries

MT-004	Critical	3d	Sec 4.1, 4.2	TenantMiddleware, FeatureAuthorizationFilter, and JWT organization_id claim
---------------	-----------------	-----------	---------------------	--

Build the request pipeline: TenantMiddleware resolves tenant from JWT, FeatureAuthorizationFilter checks feature enablement, and the login flow includes organization_id in JWT claims.

Development tasks	Unit test cases
-------------------	-----------------

• TenantMiddleware: reads organization_id claim from JWT, resolves Organization from cache/DB, creates scoped ITenantContext, injects into DI. Returns 401 if org is deactivated. • FeatureAuthorizationFilter: [RequiresFeature('MODULE_PROCUREMENT')] attribute on controllers/actions. Filter reads the org's enabled features (cached), returns 403 if feature disabled. • Update login flow: on successful authentication, add organization_id and is_super_admin claims to the JWT token • Super Admin detection: if user exists in SuperAdminUsers table, set IsSuperAdmin = true and skip tenant scoping • GET /api/tenant/current - returns current org info, list of enabled feature_codes, user's role and permissions. React sidebar consumes this on every page load to render/hide menu items. • Cache OrganizationFeatures per org_id with short TTL (e.g. 5 min) so feature toggles take effect quickly without per-request DB hits	• Request with JWT containing org_id for an active org: TenantMiddleware sets ITenantContext correctly • Request with JWT for a deactivated org returns 401 • Controller with [RequiresFeature('MODULE_MIR')] returns 403 when MIR is disabled for the org • Same controller returns 200 when MIR is enabled • Super Admin request bypasses both tenant filter and feature filter • GET /api/tenant/current returns correct enabled features for the org, matching OrganizationFeatures • Feature toggle change (Super Admin disables a feature): within 5 minutes, the next request from that org gets 403 on the disabled feature's endpoints
---	---

**Acceptance
criteria**

• Tenant resolution from JWT is reliable and performant • Feature authorization enforced at API level, not just UI • Deactivated org immediately blocked • Cache TTL ensures feature changes propagate quickly

Epic 3 - Sidebar, Role Scoping, and Workflow per Organization

React sidebar driven by org features + role permissions, org-scoped roles, and workflow definitions per organization.

MT-005

High

3d

Sec 4.1, 4.3

React sidebar rendering driven by organization features and user role

Refactor the React sidebar to dynamically render menu items based on the intersection of the organization's enabled features and the user's role permissions, consumed from GET /api/tenant/current.

Development tasks	Unit test cases
- Refactor sidebar component: on app load, call GET /api/tenant/current; store enabled features and role permissions in React context/state - Each sidebar menu item tagged with its required feature_code (e.g. Procurement menu requires MODULE_PROCUREMENT) - Rendering logic: show item only if feature_code is in enabledFeatures AND user's role has at least one permission for that module - Super Admin sidebar: show ALL items regardless of any org's feature set (Super Admin is outside org scope) - Organization Admin: show all items for features enabled for their org (Admin implicitly has all permissions for enabled features) - Regular users: most restrictive intersection of org features + role permissions - Add System Administration menu group (Organizations, Features) visible only to Super Admin	- Org with MODULE_FINANCE disabled: Finance menu items absent from sidebar for all users in that org - Same user logging into a different org that HAS MODULE_FINANCE enabled: Finance menu items appear - Super Admin sees System Administration menu group; regular Admin does not - User with Requester role in an org with MODULE_PROCUREMENT enabled: no Procurement menu (role doesn't permit it) - User with Procurement Manager role in same org: Procurement menu visible - Org Admin sees all enabled features regardless of role-level permissions
Acceptance criteria	- Sidebar dynamically reflects both org features and role permissions - No hardcoded menu visibility - everything driven by API response - Super Admin menu correctly isolated

MT-006

High

2d

Sec 6

Organization-scoped workflow definitions and default seeding

Scope all WorkflowDefinitions by organization_id. Seed default workflow templates for new orgs at creation time. Ensure approver resolution scopes to users within the same organization.

Development tasks	Unit test cases
- WorkflowDefinitions already has organization_id from MT-003. Verify the EF Core query filter correctly scopes workflow queries to the current org. - On organization creation (MT-002): seed default WorkflowDefinitions for the new org, cloned from a system template (same defaults as the original single-tenant system had for MIR, PO, etc.) - Verify IApproverResolutionService scopes user lookups to the current organization: a Supervisor step in Org A never resolves to a user in Org B - Organization Admin can view and edit their own org's workflow definitions via the existing Workflow Configuration screen (Addendum 16) - now automatically scoped - Super Admin can view workflow definitions across all organizations	- Org A and Org B can both have a MIR workflow definition with different step configurations - Submitting an MIR in Org A resolves approvers only from Org A's users - Creating a new organization auto-seeds default workflow definitions - Org Admin editing a workflow in Org A does not affect Org B's workflow for the same interface_code - Super Admin can view Org A's and Org B's workflows side by side

Acceptance criteria	• Workflows correctly isolated per organization • Default seeding on org creation gives new orgs a working workflow immediately • Cross-org approver resolution is impossible
---------------------	---

Epic 4 - Migration, Super Admin User, and Regression Verification

Data migration assigning all existing data to SCM-DEMO, Super Admin user creation, and full regression testing.

MT-007

Critical

2d

Sec 7

Designate Super Admin, migrate existing data, and create the migration s

Create the SuperAdminUsers table, designate one existing admin as Super Admin, and verify the full migration path from Steps 1-7 of FSD Section 7.1.

Development tasks	Unit test cases
• Create SuperAdminUsers table in tenant_schema: user_id FK (to auth_schema.Users), created_at. No organization_id on this table. • Migration script: designate the first existing System Admin user as Super Admin by inserting into SuperAdminUsers • Update this user's JWT generation to include is_super_admin = true claim • Verify all existing data across all schemas has organization_id = SCM-DEMO UUID (already handled by DEFAULT in MT-003, but explicitly verify via count queries) • Verify all existing roles have organization_id = SCM-DEMO • Verify all existing WorkflowDefinitions have organization_id = SCM-DEMO • Create a one-time migration verification script that counts rows per table and confirms organization_id is populated on every row with no nulls	• Super Admin user can log in and sees the System Administration menu • Super Admin can create a new organization via POST /api/system/organizations • All existing users (except Super Admin) are scoped to SCM-DEMO and see only SCM-DEMO data • Row count verification: every tenant-scoped table has zero rows with NULL organization_id • Existing Super Admin user appears in SuperAdminUsers table • Login as a regular SCM-DEMO user shows identical experience to pre-migration (no visible change)
Acceptance criteria	• Super Admin correctly designated and functional • All existing data verifiably belongs to SCM-DEMO • Zero null organization_id rows across all tables

MT-008

Critical

3d

Sec 7 (Step 8)

Full regression test suite and multi-tenant integration tests

Run the complete existing test suite to verify zero-change regression for SCM-DEMO, then add multi-tenant-specific integration tests covering cross-org isolation.

Development tasks	Unit test cases
• Run all existing unit and integration tests without modification - every test must pass because SCM-DEMO has all features and all data • Add integration test: create Org B via Super Admin, create a user in Org B, create a Product in Org B, verify Org A cannot see Org B's product • Add integration test: user in Org A creates a PO, user in Org B queries POs, Org B sees zero results (not Org A's PO) • Add integration test: disable MODULE_FINANCE for Org B, verify Org B user gets 403 on /api/supplier-payments • Add integration test: Org Admin in Org A cannot access /api/system/organizations (Super Admin only) • Add integration test: concurrent operations in Org A and Org B do not interfere with each other's data • Add integration test: deactivating Org B immediately prevents Org B users from authenticating • Performance test: verify query filter on organization_id uses the index and does not degrade query performance vs pre-migration baseline	• 100% of existing test suite passes without modification (zero-change regression for SCM-DEMO) • Cross-org data isolation: Org A cannot see Org B's suppliers, POs, GRNs, invoices, or any other business data • Feature gating: disabled feature returns 403 at API level for Org B • Role isolation: Org A's roles do not appear in Org B's role list • Workflow isolation: submitting a document in Org A resolves only Org A's approvers • Org deactivation: login attempt for Org B user returns 401 immediately after deactivation • Query performance: SELECT with organization_id filter uses index scan, not table scan

Acceptance criteria	• Zero existing test failures (proof that SCM-DEMO behaves identically to pre-migration) • Cross-org isolation proven across all major entity types • Performance not degraded by the additional WHERE clause
---------------------	---

3. Sequencing Notes

Dependencies follow the natural build order: foundation first, then isolation, then UI, then migration verification.

Dependency	Reason
MT-001 before everything else	All tasks depend on tenant_schema and the Organizations/Features entities existing.
MT-001 before MT-002	CRUD API operates on entities created by MT-001.
MT-003 before MT-004	Middleware reads organization_id which must exist in tables first.
MT-003 and MT-004 before MT-005	Sidebar logic depends on both the data scoping and the feature authorization filter.
MT-003 before MT-006	Workflow scoping depends on organization_id on WorkflowDefinitions.
MT-001 through MT-006 before MT-007	Super Admin designation and migration verification need all infrastructure in place.
MT-007 before MT-008	Regression tests verify the migrated state from MT-007.
MT-001+MT-002 can parallel MT-003	Org CRUD and schema migration can be developed simultaneously if coordinated.

- End of Document -